- 01 — System Overview
  - What Is This System?
  - High-Level Architecture
  - Key Concepts
    - 🤖 Agents
    - 🧠 LangGraph
    - 🔧 MCP (Model Context Protocol)
    - 📊 Gemini LLM
    - 📝 Output
    - 🔄 Polling
  - Technology Stack
  - What's Next?

01 — System Overview

Welcome! This document gives you a bird's-eye view of the **Agentic Analysis System** — what it is, why it exists, and how all the pieces fit together.

What Is This System?

The Agentic Analysis System is an **AI-powered data analysis platform** for an ore dressing (mineral processing) factory. It lets plant operators and managers ask questions in plain language — like *"Compare the ore feed rate across all mills for the last 72 hours"* — and get back a full professional report with charts, tables, statistics, and recommendations.

Behind the scenes, a team of **AI agents** collaborates to:

1. Load the right data from the factory's PostgreSQL database
2. Perform statistical analysis and generate charts
3. Review the work for quality
4. Write a comprehensive Markdown report

All of this happens automatically, orchestrated by a **multi-agent graph**.

High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                        BROWSER (Next.js)                         │
│                                                                  │
│   ┌──────────────┐     ┌──────────────┐     ┌──────────────────┐ │
│   │  Chat UI     │     │  Zustand     │     │  Markdown        │ │
│   │  (page.tsx)  │◄──►│  Store       │     │  Renderer        │ │
│   │              │     │  (chat-store)│     │  (ReactMarkdown) │ │
│   └──────────────┘     └──────────────┘     └──────────────────┘ │
│          │                    │                                  │
│          │    POST /analyze   │  GET /status (polling)           │
│          └────────────────────┼──────────────────────────────────┘
│                               ▼                                  │
├──────────────────────────────────────────────────────────────────┤
│                     FASTAPI SERVER (:8000)                       │
│                                                                  │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │  api_endpoint.py (REST API)                              │   │
│   │  POST /api/v1/agentic/analyze   → starts background task│   │
│   │  GET  /api/v1/agentic/status/id → returns progress      │   │
│   │  GET  /api/v1/agentic/reports/  → serves files          │   │
│   └──────────────────────────────────────────────────────────┘   │
│                  │                                               │
│                  │  MCP Client (client.py)                       │
│                  ▼                                               │
├──────────────────────────────────────────────────────────────────┤
│                     MCP SERVER (:8003)                           │
│                                                                  │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │  server.py  (Starlette + Streamable HTTP)                │   │
│   │                                                          │   │
│   │  Tools:                                                  │   │
│   │    • query_mill_data     — Load mill sensor data         │   │
│   │    • query_combined_data — Load mill + ore quality       │   │
│   │    • get_db_schema       — Inspect database structure    │   │
│   │    • execute_python      — Run analysis code (pandas)    │   │
│   │    • list_output_files   — Check generated charts        │   │
│   │    • write_markdown_report — Write the final report      │   │
│   │    • set_output_directory  — Set per-analysis output dir │   │
│   └──────────────────────────────────────────────────────────┘   │
│                  │                                               │
│                  ▼                                               │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │  PostgreSQL Database                                     │   │
│   │  Schema: mills                                           │   │
│   │  Tables: MILL_01..MILL_12 (minute-level sensor data)     │   │
│   │          ore_quality (lab measurements)                  │   │
│   └──────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────┘
```

Key Concepts

🤖 Agents

AI specialists that each handle one job. There are four:

- **Data Loader** — fetches data from PostgreSQL
- **Analyst** — runs Python code for EDA, SPC, charts
- **Code Reviewer** — validates outputs, fixes errors
- **Reporter** — writes the final Markdown report

🧠 LangGraph

The orchestration framework (from LangChain) that connects agents into a directed graph. It handles state, routing, and tool calls.

🔧 MCP (Model Context Protocol)

A protocol for connecting AI models to external tools. The MCP Server exposes tools (query data, run Python, etc.) and the MCP Client wraps them for LangChain.

📊 Gemini LLM

Google's `gemini-3.1-flash-lite-preview` model powers all agents. Each agent gets a different system prompt and different tool bindings.

📝 Output

Each analysis gets a unique ID (e.g. `51329fe7`) and its own output folder at `python/agentic/output/{id}/`. This folder contains:

- `.md` — Markdown report
- `.png` — Chart images

🔄 Polling

The frontend submits an analysis request (POST), then polls the status endpoint (GET) every 4 seconds until the analysis completes or fails.

Technology Stack

Layer	Technology
Frontend	Next.js, React, TypeScript, TailwindCSS
State Management	Zustand
Markdown Rendering	react-markdown + remark-gfm
Backend API	FastAPI (Python)
Agent Orchestration	LangGraph (LangChain)
LLM	Google Gemini (via langchain-google-genai)
Tool Protocol	MCP (Model Context Protocol)
Database	PostgreSQL
Data Analysis	pandas, numpy, seaborn, matplotlib, scipy

What's Next?

- **02 — Backend Architecture** — How FastAPI and the MCP server work
- **03 — Multi-Agent Graph** — The agent pipeline in detail
- **04 — MCP Tools** — Every tool explained
- **05 — Frontend** — The chat UI and state management
- **06 — Data Flow** — End-to-end request lifecycle
- **07 — Code Reference** — Key functions documented